

# Frontend React Notes

Commands, Examples, Project Structure & Base Application

## Contents

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>What is React?</b>                                       | <b>2</b> |
| <b>2</b> | <b>Commands for React Development</b>                       | <b>2</b> |
| <b>3</b> | <b>React Project Structure</b>                              | <b>2</b> |
| <b>4</b> | <b>Analysis of main.jsx</b>                                 | <b>3</b> |
| <b>5</b> | <b>JavaScript Basics</b>                                    | <b>4</b> |
| <b>6</b> | <b>TypeScript Basics</b>                                    | <b>6</b> |
| <b>7</b> | <b>Frontend Fundamentals: JavaScript, React, TypeScript</b> | <b>7</b> |

# What is React?

React is a JavaScript library for building user interfaces with reusable components.

- $\text{React} = \text{UI} = f(\text{state})$ . When state changes UI updates automatically.

## Commands for React Development

### Create React project using Vite

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

### Common npm Commands

```
npm install
npm run dev
npm run build
npm run preview
```

## React Project Structure

```
src/
  main.jsx          <-- mounts React into <div id="root">
  App.jsx           <-- main application logic

  components/       <-- optional reusable components
    Header.jsx
    TodoList.jsx

  App.css           <-- component-specific styles
  index.css         <-- global baseline styles
```

React projects follow a simple pattern:

- `main.jsx` starts the app.
- `App.jsx` contains the user interface.
- `components/` stores reusable UI pieces.

## Analysis of `main.jsx`

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

`main.jsx` loads the React application and injects it into:

```
<div id="root"> ... </div>
```

Everything drawn by React appears inside this container.

# JavaScript Basics

JavaScript is the core language used to build dynamic web applications.

## Variables

```
let name = "Anna";  
const age = 30;
```

- `let` value can change
- `const` value cannot change

## Functions

```
function add(a, b) {  
  return a + b;  
}  
  
const multiply = (a, b) => a * b;
```

Arrow functions are widely used in React.

## Arrays and Looping

```
const numbers = [1, 2, 3];  
  
numbers.forEach(n => console.log(n));  
  
const doubled = numbers.map(n => n * 2);
```

`forEach` perform action  
`map` create new array

## Objects

```
const user = {  
  name: "Anna",  
  age: 30  
};
```

## Async / Await

```
const fetchData = async () => {  
  const response = await fetch("url");  
  const data = await response.json();  
  console.log(data);  
};
```

`async` defines an asynchronous function  
`await` pauses execution until result is ready

# TypeScript Basics

TypeScript is JavaScript with static typing.

## Basic Types

```
let name: string = "Maria";  
let age: number = 30;  
let isStudent: boolean = true;
```

## Functions with Types

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

## Interfaces

```
interface User {  
    name: string;  
    age: number;  
}  
  
const user: User = {  
    name: "Maria",  
    age: 30  
};
```

## Arrays of Objects

```
const users: User[] = [  
    { name: "Maria", age: 30 },  
    { name: "John", age: 25 }  
];
```

# Frontend Fundamentals: JavaScript, React, TypeScript

A modern frontend application is built by combining three layers:

- JavaScript logic and behavior
- React user interface (UI)
- TypeScript type safety and structure

## JavaScript: Core Concepts

JavaScript is responsible for the logic of the application. It defines variables, functions, and how data is manipulated.

```
const name = "Maria";
let count = 0;

function add(a, b) {
  return a + b;
}

const greet = (name) => {
  return "Hello " + name;
};
```

Functions define reusable behavior. Arrow functions are commonly used in React.

## Asynchronous Code and API Calls

Frontend applications often need data from a server. This is done using asynchronous code.

The browser does not wait for the server response automatically.

```
const fetchData = async () => {
  const response = await fetch("url");
  const data = await response.json();
  console.log(data);
};
```

`async` defines a function that contains asynchronous operations.  
`await` pauses execution until the result is available.

The typical sequence is:

1. send request with `fetch`
2. receive response
3. convert to JSON
4. use the data

## React: Components and UI

React builds the user interface using components.

A component is a function that returns UI (written in JSX).

```
function App() {  
  return <h1>Hello</h1>;  
}
```

JSX allows mixing HTML with JavaScript. Values are inserted using curly braces: { }.

```
function App() {  
  const name = "Maria";  
  return <h1>Hello {name}</h1>;  
}
```

## State and Interaction

State represents data that changes over time.

React provides `useState` to store and update values.

```
const [count, setCount] = useState(0);
```

```
<button onClick={() => setCount(count + 1)}>  
  Increase  
</button>
```

Events such as clicks trigger state updates. When state changes, the UI re-renders automatically.



## Rendering Lists

```
const users = ["Anna", "John"];

return (
  <div>
    {users.map((user) => (
      <p key={user}>{user}</p>
    ))}
  </div>
);
```

`map()` transforms data into UI elements. Each element must have a unique `key`.

## Fetching Data in React

```
useEffect(() => {
  const loadData = async () => {
    const response = await fetch("url");
    const data = await response.json();
    setUsers(data);
  };

  loadData();
}, []);
```

`useEffect` runs when the component loads.  
It is used to:

- fetch data
- initialize state

## TypeScript: Adding Structure

TypeScript adds types to JavaScript.

It defines what kind of data is expected and prevents incorrect usage.

```
let age: number = 30;
let name: string = "Anna";
```

```
interface User {  
  id: number;  
  name: string;  
}  
  
const [users, setUsers] = useState<User[]>([]);
```

TypeScript ensures that data follows a defined structure.  
This is especially useful when working with API responses.

## Combined Workflow

A typical frontend flow is:

1. define data structure (TypeScript)
2. create component (React)
3. fetch data (JavaScript async/await)
4. store data in state
5. render data using JSX

These steps are repeated in most frontend applications.